Mode d'emploi de la carte électronique pour le lickport (détecteur/valve)

Tous les documents et codes sont disponibles sur GitHub : https://github.com/ShulzLab/Lickport

Sommaire :

A. Photo de la « carte lickport »
B. Objectifs de la carte
C. Schéma simplifié de la connectique
D. En pratique, quel fils brancher où ?
E. Boite désignée pour protéger la carte électronique
F. Exemple de code Arduino (avec la Giga R1 wifi)
G. Annexes

A. Photos de la « carte lickport »

Photo de la carte sans les composants

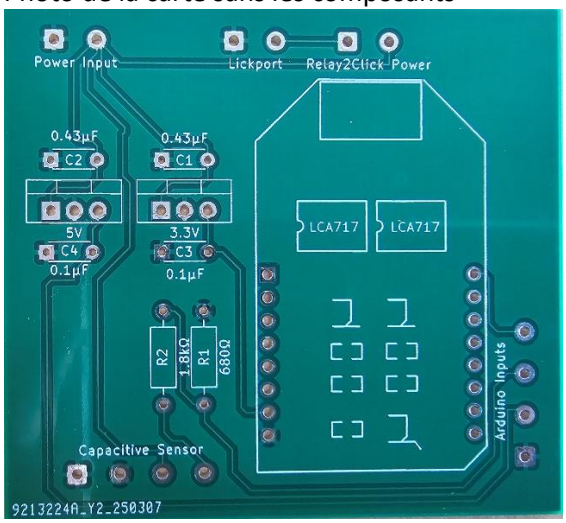
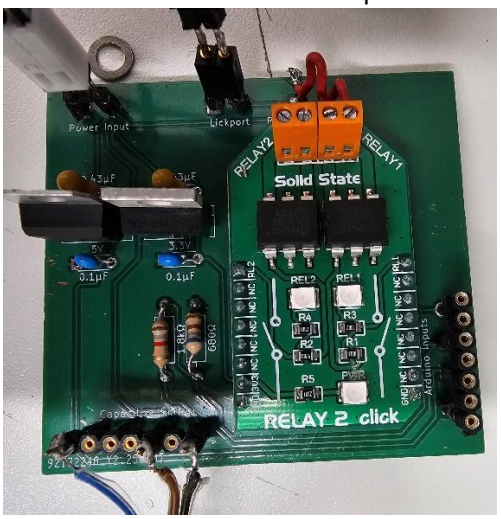


Photo de la carte avec les composants



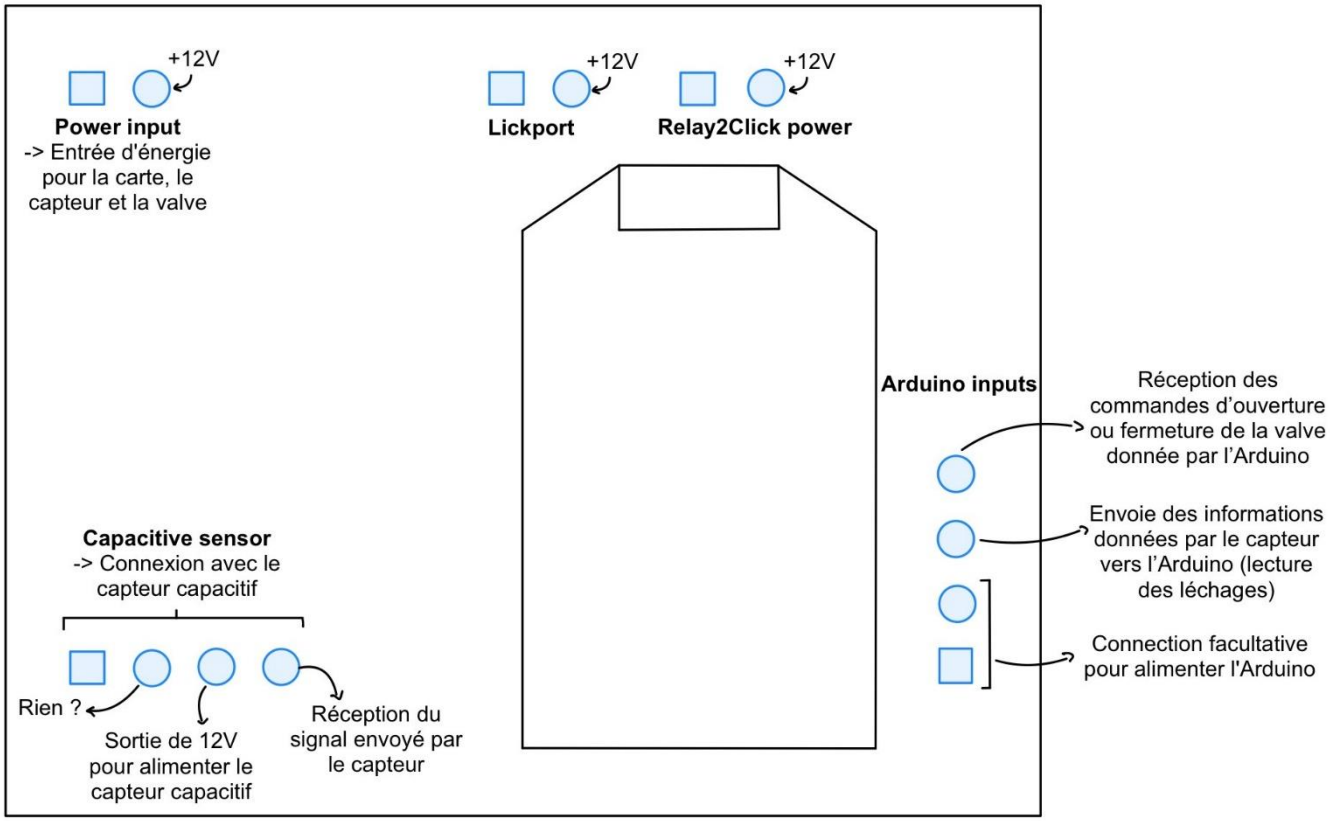
B. Objectifs de la carte

Cette carte permet de :

- **fournir une alimentation 12V** au capteur capacitif et à la valve
- **recevoir les informations de léchage** issues du capteur capacitif et les envoyer vers l'Arduino en 3,3V théorique
- **ouvrir la valve** suite à une commande de l'Arduino (codage de l'information grâce au « relay2click », cad. la partie surélevée de carte électronique)

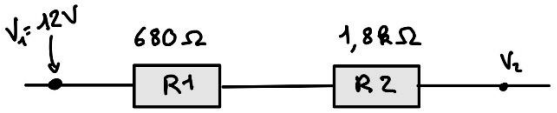
C. Schéma simplifié de la connectique

Note : les connexions carrés correspondent aux grounds (bornes négatives)



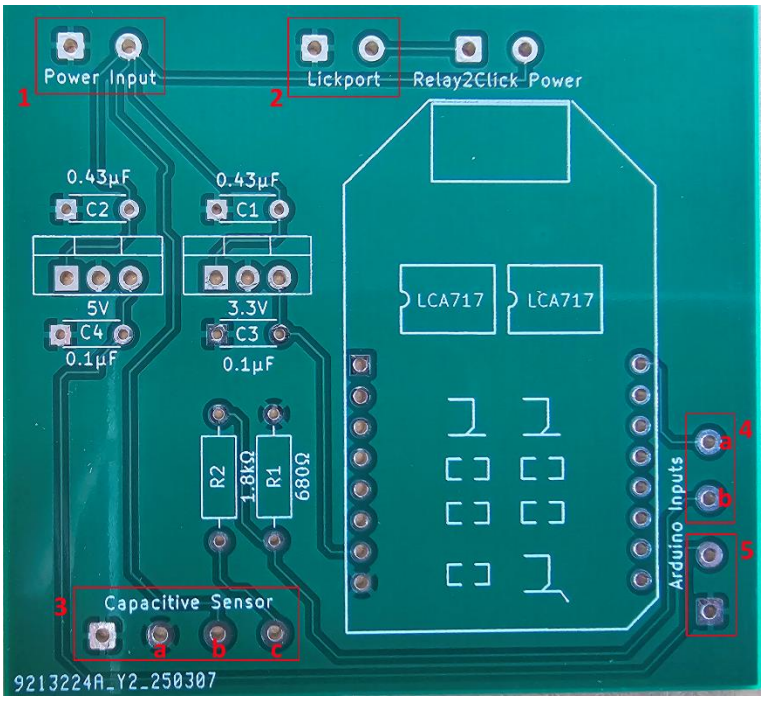
(Le schéma complet est disponible en annexe)

Remarque :
Choix des résistances de R1 = 680Ω et R2 = 1,8kΩ afin d'avoir une sortie vers l'Arduino à 3,3V



$$V_2 = V_1 \times \frac{R_1}{R_1 + R_2} = 12 \times \frac{680}{680 + 1800} = 3,3\ V$$

D. En pratique, quel fils brancher où ?



1. Alimentation 12V
2. Fils électrique de la valve
3. Fils électriques du capteur capacitif *(La borne carré correspond au ground)*
   a. ⚠ Ne sert pas
   b. Envoie de puissance 12V vers le capteur capacitif (ici fils marron)
   c. Lecture du signal provenant du capteur capacitif (ici fils noir)
4. Connexion avec l'Arduino obligatoire
   a. Sortie de l'Arduino vers la « carte lickport » pour ouvrir la valve (ici fils violet, à connecter au bon pin de l'Arduino défini dans le code)
   b. Entrée vers l'Arduino depuis la « carte lickport » pour lire les léchages (envoie des informations en 3,3V théorique – voir le schéma simplifié pour l'explication des 3,3V)
5. Connexion avec l'Arduino facultative afin d'alimenter directement l'Arduino sans avoir à connecter l'Arduino à l'ordinateur

⚠ Les pores femelles de la « carte lickport » sont plus petits que la normale (cad. ceux de l'Arduino), il faut donc si besoin souder soit même de nouveaux embouts males sur des câbles classiques.

E. Boite désignée pour protéger la carte électronique

La boite a été faite sur-mesure pour éviter toute tension au niveau des soudures et des câbles liés à la carte électronique (comme les connexions sont un peu particulières).
La boite a été conçue à l'aide du logiciel Tinkercad.

Voici les liens pour accéder aux designs :

- Design de la boite : https://www.tinkercad.com/things/hbnSuq6Cchr-boite-carte-lickport-v1-avec-accroches/edit?sharecode=Nxq6ZqytI9hEeUHwjgE4FO9mnY3RvXSJX_H6KoXoO60
- Design du couvercle : https://www.tinkercad.com/things/5hVfECfCvXz-couvercle-boite-carte-lickport-v1?sharecode=sEUgVh3V9k8983TuvGKIf2XeCjM6gngVLM33MLh_zIU

Les documents .stl ont également été mis sur GitHub.

F. Exemple de code Arduino (avec la Giga R1 wifi)

Stimulation tactile → Ouverture de la valve pendant 2s
Il s'agit d'un prototype, il faut adapter le code en fonction de la tâche effectuée et des données voulues.
Voir sur GitHub pour la version à copier direct.

```
/*
 * Lick Port System
 * Detects licks and delivers water as reward
 */

const int waterValvePin = 8;     // Digital pin controlling water valve
const int lickSensorPin = 13;        // Input pin for lick sensor

byte LickState = LOW;                // State of lick sensor without touch - Can be "HIGH"
regarding to the type of capacitive sensor used

// Time parameters
const unsigned long waterDuration = 2000;    // Duration for opening water valve (us) =>
Parameter to adjust based on the desired amount of water delivered

unsigned long lastLickTime = 0;   // Last time a lick was detected
unsigned long timerStart = 0;     // Start time for water delivery
unsigned long currentStateChangeTime = 0; // Time when the state last changed
unsigned long lastActivityTime = 0; // Last time any activity was detected

unsigned long waterStartTime = 0; // Time opening water valve (us)
unsigned long waterEndTime = 0;   // Time closing water valve (us)

// State machine variables
enum class WaterState : uint8_t{WAITING,  DISPENSING};   // Define possible states for the
water supply system
WaterState currentWaterState = WaterState ::WAITING;


unsigned long lickTime[100];  // Array to store lick timestamps => /!\ Warning: this array
has a fixed size, consider using a dynamic data structure for larger datasets
unsigned long waterTime[100]; // Array to store water delivery timestamps

int lickEventCount = 0;            // Counter for number of licks
int waterEventCount = 0;           // Counter for number of water deliveries


//Setup, initialization of pins and timestamps
void setup() {
```

```
    Serial.begin(9600);

    pinMode(waterValvePin, OUTPUT);

    pinMode(lickSensorPin, INPUT_PULLUP);

        // Initialize event counters
    memset(lickTime, 0, sizeof(lickTime));
    memset(waterTime, 0, sizeof(waterTime));

    // Reset state machine
    currentWaterState = WaterState::WAITING;
    lastActivityTime = millis();

}

// Main loop to handle lick detection and water delivery
void loop() {
    unsigned long currentTime = millis();

    switch(currentWaterState) {
        case WaterState::WAITING:
            // Check for lick detection
            if(digitalRead(lickSensorPin) == HIGH && ((currentTime - lastLickTime >= 50) ||
lickEventCount == 0)) {
              // HIGH = lick detected
                lastLickTime = currentTime;

                lickTime[lickEventCount] = currentTime;
                lickEventCount++;

                Serial.println("Lick detected!");

                currentStateChangeTime = currentTime; // Start the delay timer
                currentWaterState = WaterState::DISPENSING;      // Change state to LICKING

                waterStartTime = currentTime;
                digitalWrite(waterValvePin, HIGH);
                waterTime[waterEventCount] = currentTime;
                waterEventCount++;
                Serial.println("Water dispensing started");
            }
            break;

        case WaterState::DISPENSING:
        // Dispense water after a lick is detected for a certain duration

            if(currentTime - waterStartTime >= waterDuration) {
                digitalWrite(waterValvePin, LOW);  // Close water valve
```

```
                waterEndTime = currentTime;
                Serial.println("Water delivery ended");

                currentWaterState = WaterState ::WAITING;    // Change state back to WAITING
            }
            break;
    }
}
```

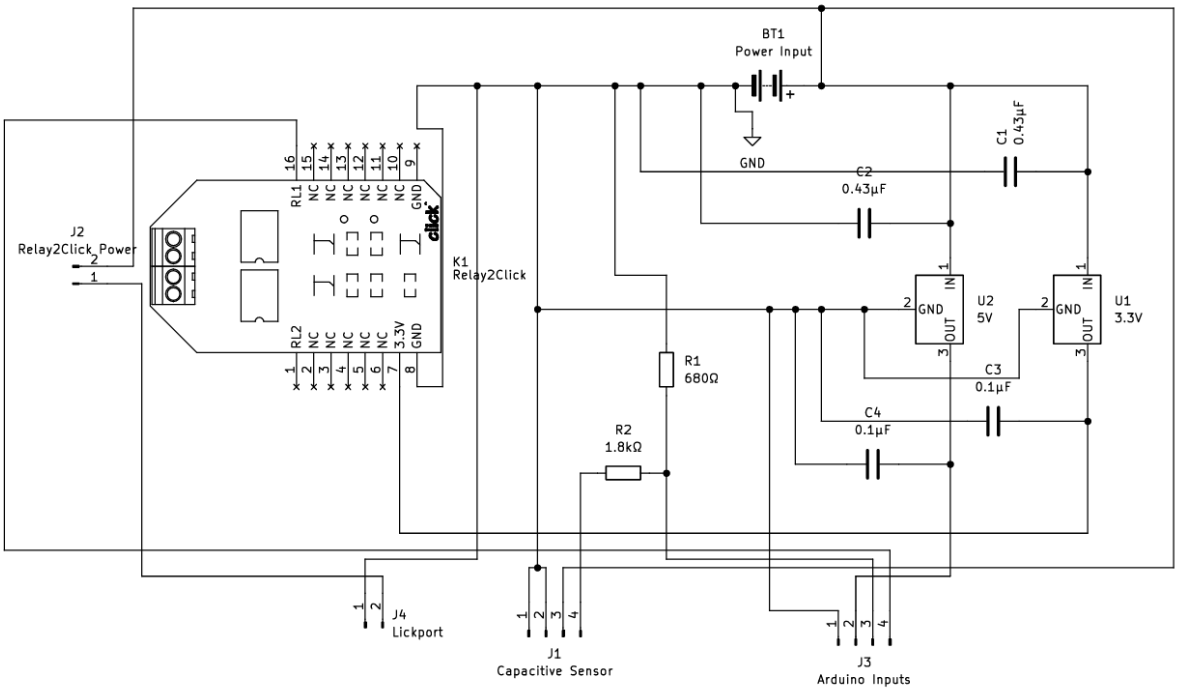
Code pour la calibration du lickport

Il faut entrer dans le moniteur série la commande « START x » avec x le nombre de secondes d'ouverture de la valve voulues.
Le programme permet de faire couler 100 gouttes d'eau, avec un espacement de 2s entre chaque ouverture de valve.
Voir sur GitHub pour la version à copier direct.

```
/*
   Valve Pulse System
   Command format:
   START X

   Example:
   START 3   -> 3 seconds open duration
*/

const int waterValvePin = 8;     // Digital pin controlling water valve => To adjust based
on the actual wiring of the system

// Time parameters
const unsigned long closedDuration = 2000; // 2 second closed
const int totalPulses = 100;

unsigned long openDuration = 2000; // default 2 seconds (ms)

unsigned long stateStartTime = 0;
int pulseCount = 0;

// State machine variables
enum class ValveState : uint8_t{IDLE, OPEN,  CLOSED, FINISHED};   // Define possible states
of the valve control system
ValveState currentState = ValveState ::IDLE;

String inputCommand = "";


void setup() {
```

```
    Serial.begin(9600);
    pinMode(waterValvePin, OUTPUT);
    digitalWrite(waterValvePin, LOW);   // Ensure valve is closed at startup
    Serial.println("System ready.");
    Serial.println("Type: START X  (X = seconds open)");
}


void loop() {
    unsigned long currentTime = millis();

    // ----- SERIAL COMMAND -----
    while (Serial.available()) {
        char c = Serial.read();

        if (c == '\n') {                    // End of command after newline
            inputCommand.trim();            // Remove any leading/trailing whitespace

            if (inputCommand.startsWith("START")) {     // Verify that the command starts
with "START"

                int spaceIndex = inputCommand.indexOf(' ');         // Find the space
separating command and parameter
                if (spaceIndex > 0) {
                    int seconds = inputCommand.substring(spaceIndex + 1).toInt(); //
Extract the number of seconds and convert to integer

                    if (seconds > 0) {
                    openDuration = (unsigned long)seconds * 1000;       // Convert seconds
to milliseconds

                    pulseCount = 0;                           // Reset pulse count for new
sequence
                    stateStartTime = currentTime;
                    currentState = ValveState::OPEN;

                    Serial.print("Sequence started with ");
                    Serial.print(seconds);
                    Serial.println(" seconds open duration.");
                    }
                }
            }

            inputCommand = "";          // Clear the command buffer for the next command
        }
        else {
        inputCommand += c;              // If there is no newline, keep building the
command string
        }
```

```
    }

    // ----- STATE MACHINE -----
    switch (currentState) {

        case ValveState::IDLE:          // Do nothing, waiting for command
            break;

        case ValveState::OPEN:
            digitalWrite(waterValvePin, HIGH);      // Open water valve

            if (currentTime - stateStartTime >= openDuration) {         // If the opening
time has expired: Close the valve
                digitalWrite(waterValvePin, LOW);
                stateStartTime = currentTime;
                pulseCount++;

                Serial.print("Pulse ");
                Serial.print(pulseCount);
                Serial.println(" completed");

                if (pulseCount >= totalPulses) {                // If the total number of
pulses has been reached, finish the sequence
                    currentState = ValveState::FINISHED;
                    Serial.println("Sequence finished.");
                } else {
                    currentState = ValveState::CLOSED;          // Otherwise, go to closed
state before the next pulse
                }
            }
        break;

        case ValveState::CLOSED:
            if (currentTime - stateStartTime >= closedDuration) {   // After the closed
duration, go back to open state for the next pulse
                stateStartTime = currentTime;
                currentState = ValveState::OPEN;
            }
            break;

        case ValveState::FINISHED:
            digitalWrite(waterValvePin, LOW);
            break;
    }
}
```

G. Annexe : Schéma électronique de Jimmy

Voir sur GitHub pour la version Kicad.